

LAB : *Le Model Context Protocol (MCP)*

Table des matières

PARTIE 1 : Intégration d'un serveur MCP local avec chargement dynamique de tools, ressources et prompts.....	2
MCP local server via stdio	2
Récupération dynamique des tools.....	2
MCP.....	2
Prompt dynamique côté serveur MCP	3
Agent LLM modulaire avec serveur MCP local (tools, ressources et prompts dynamiques)	3
PARTIE 2 : Agent LLM avec outil MCP de temps (time server) pour interrogation de l'heure en zone spécifique	4
Configuration et initialisation d'un client MCP pour un serveur de temps avec gestion de timezone	4
Récupération dynamique des tools.....	4
Création et exécution d'un agent LLM avec outils MCP pour la consultation du temps local.....	5
PARTIE 3 : Agent pour se connecter à un serveur MCP distant via HTTP streaming	6

PARTIE 1 : Intégration d'un serveur MCP local avec chargement dynamique de tools, ressources et prompts

MCP local server via stdio

```
import asyncio
```

```

from langchain.agents import create_agent
from langchain_mcp_adapters.client import
MultiServerMCPClient from langchain.agents import
create_agent
from langchain.messages import HumanMessage
from langchain_ollama import ChatOllama

async def main():
    client = MultiServerMCPClient(
        {
            "local_server": {
                "transport": "stdio",
                "command": "python",
                "args": ["resources/mcp_local_server.py"],
            }
        }
    )

```

Récupération dynamique des tools

```

# get tools
tools = await client.get_tools()

```

Resources MCP

```

# get resources
resources = await client.get_resources("local_server")

```

SMA et IAD Master SDIA Prof. RETAL SARA

Prompt dynamique côté serveur MCP

```

# get prompts
prompt = await client.get_prompt("local_server", "prompt")
prompt = prompt[0].content

```

Agent LLM modulaire avec serveur MCP local (tools, resources et prompts dynamiques)

```

# Initialiser le modèle Ollama

```

```

model = ChatOllama(
    model="llama3.2:3b", # ou mistral, gemma, etc.
    temperature=0
)

agent = create_agent(
    model=model,
    tools=tools,
    system_prompt=prompt
)
config = {"configurable": {"thread_id": "1"}}

response = await agent.ainvoke(
    {"messages": [HumanMessage(content="Tell me about the langchain-mcp-adapters library)]}],
    config=config
)
print(response)

asyncio.run(main())

```

N.B : *uv run --active python agentMCP.py pour exécuter*
SMA et IAD Master SDIA Prof. RETAL SARA

PARTIE 2 : Agent LLM avec outil MCP de temps (time server) pour interrogation de l'heure en zone spécifique

Configuration et initialisation d'un client MCP pour un serveur de temps avec gestion de timezone

```

import asyncio
from langchain.agents import create_agent
from langchain_mcp_adapters.client import
MultiServerMCPClient from langchain.agents import
create_agent
from langchain.messages import HumanMessage
from langchain_ollama import ChatOllama

async def main():
    client = MultiServerMCPClient(
        {
            "time": {
                "transport": "stdio",
                "command": "uvx",

```

```

        "args": [
            "mcp-server-time",
            "--local-timezone=America/New_York"
        ]
    }
}
)

```

Récupération dynamique des tools

```

# get tools
tools = await client.get_tools()

```

SMA et IAD Master SDIA Prof. RETAL SARA

Création et exécution d'un agent LLM avec outils MCP pour la consultation du temps local

```

# Initialiser le modèle Ollama
model = ChatOllama(
    model="llama3.2:3b", # ou mistral, gemma, etc.
)

agent = create_agent(
    model=model,
    tools=tools,
)
question = HumanMessage(content="What time is it in Japan")

response = await agent.ainvoke(
    {"messages": [question]}
)

print(response['messages'][-1].content)

asyncio.run(main())

```

N.B : *uv run --active python agentMCPTIME.py pour exécuter*

SMA et IAD Master SDIA Prof. RETAL SARA

PARTIE 3 : Agent pour se connecter à un serveur MCP distant via HTTP streaming

```

import asyncio
from langchain.agents import create_agent
from langchain_mcp_adapters.client import
MultiServerMCPClient from langchain.agents import
create_agent
from langchain.messages import HumanMessage
from langgraph.checkpoint.memory import InMemorySaver
from dotenv import load_dotenv

async def main():

    client = MultiServerMCPClient(
        {
            "travel_server": {
                "transport": "streamable_http",
                "url": "https://mcp.kiwi.com"
            }
        }
    )

    # get tools
    tools = await client.get_tools()

    load_dotenv()

    agent = create_agent(
        "gpt-5-nano",
        tools=tools,
        checkpoint=InMemorySaver(),
        system_prompt="You are a travel agent. No follow up
        questions." )
    config = {"configurable": {"thread_id": "1"}}

    response = await agent.ainvoke(
        {"messages": [HumanMessage(content="Get me a direct flight from Rabat to
        Agadir on August 31st")]},
        config
    )

    print(response)

asyncio.run(main())

```